


# TP3 · Cafetería Origen: menú de mesa

Programación IV · TUP · UTN FRBB · 19 de oct de 2026 · @Sergio Antozzi

## Contexto

A Valentina le gustó tanto la API que ahora quiere que los clientes pidan sin hacer fila. En cada mesa de Origen va a haber un código QR: al escanearlo se abre el menú en el celular, el cliente arma su pedido y lo envía directo a la API.

Cada grupo lo construye sobre **la API que entregó en el Parcial I**, con los cambios mínimos que pide este enunciado. A Valentina en esta etapa, le interesa que funcione mas que la apariencia.

## Objetivo

Aplicar la teoría de frontend y testing ( usando TDD ) en una aplicación que consume una API real: la propia.

# Qué tiene que hacer el menú

El menú tiene que cumplir estas nueve funcionalidades, todas contra la API del grupo.

1. **Mesa desde el QR.** El menú se abre con `?mesa=N` (por ejemplo `/?mesa=4`) y muestra la mesa en el encabezado. Sin `?mesa`, o con un valor inválido, el pedido es para retirar en la barra.
2. **Menú cargado de la API.** Los productos salen de `GET /productos`. Mientras carga se ve un indicador; si falla, un mensaje claro y un botón para reintentar.
3. **Café en tres pasos.** El cliente elige bebida (expreso o filtrado), tamaño (pocillo, mediano o taza grande) y origen (Colombia, Etiopía o Brasil).
    - Cada origen muestra su descripción: acidez, cuerpo y notas.
    - Con bebida y tamaño elegidos, cada origen muestra su precio. Ejemplo: expreso mediano de Colombia, $7.360.
    - El botón para agregar queda deshabilitado hasta completar las tres elecciones.
4. **Pastelería.** Muffin de chocolate y porción de torta selva negra, cada uno con su precio y un botón para agregarlo.
5. **Carrito.** Muestra cada producto con su cantidad y subtotal; permite sumar, restar y quitar; muestra el subtotal general. Con el carrito vacío no se puede confirmar.
6. **Confirmar el pedido.** Envía `POST /pedidos` con la mesa y los ítems. Después muestra el número de pedido, el estado y el total **que devolvió la API**.
7. **Seguir el pedido.** Desde la confirmación, el cliente consulta el estado con `GET /pedidos/:id` sin recargar la página.
8. **Errores.** Si la API rechaza el pedido, se muestra el mensaje que devolvió y el carrito no se pierde.
9. **Pensado para el celular.** Funciona bien en una pantalla de 375 px de ancho, sin scroll horizontal.

# Cambios en la API del Parcial I

Antes de empezar el front, la API de cada grupo tiene que tener estos cambios, cada uno con su test.

| Cambio | Por qué lo necesita el front |
|---|---|
| `GET /productos` y `GET /productos/:id` devuelven `precio` (el resultado de `calcularPrecio()`) y `tipo` (`expreso`, `filtrado` o `pasteleria`) | Muestra el precio del dominio sin recalcularlo y distingue un expreso de un filtrado sin leer el nombre |

| | |
|---|---|
| `POST /pedidos` acepta `mesaId` opcional y lo devuelve en el pedido | El pedido llega a la mesa correcta |
| `GET /pedidos/:id` devuelve el pedido con su estado, sus ítems y su total | Seguimiento del pedido |
| Todos los errores responden `{ "error": "mensaje" }` | El front muestra el mensaje tal cual |
| Una mesa o un producto inexistente responde 400 o 404, nunca 500 | El cliente ve un mensaje útil |
| CORS habilitado, o un proxy de Vite hacia la API | El navegador bloquea los pedidos a otro puerto |
| Con `NODE_ENV=test`: `POST /__test__/reset` y `POST /__test__/seed` | Los E2E arrancan siempre con los mismos datos, como en el apunte de E2E |

Los nombres de los campos pueden variar entre grupos; la información no. Cada grupo documenta su contrato en el README. Como referencia, un producto:

```
{
  "id": 5,
  "tipo": "expreso",
  "nombre": "Expreso Mediano",
  "tamanio": "Mediano",
  "gramosCafe": 27,
  "variedad": {
    "nombre": "Colombia",
    "descripcion": "Acidez media, cuerpo equilibrado y notas cítricas y florales"
  },
  "precio": 7360
}
```

Y el cuerpo de `POST /pedidos` para la mesa 4, con dos expresos medianos de Colombia y un muffin. La API responde `201` con el pedido en estado `pendiente` y un total de 17220:

```
{
  "mesaId": 4,
  "items": [
    { "productoId": 5, "cantidad": 2 },
    { "productoId": 19, "cantidad": 1 }
  ]
}
```

# Requisitos técnicos

El front vive en una carpeta `frontend/` del mismo repositorio del parcial y respeta estas reglas:

- **Stack:** Vite + React + TypeScript con `"strict": true` y sin `any`.
- **Versiones fijadas** en `package.json`: Vite 7, Vitest 3, MSW 2 y Zod 4. El apunte de integración usa MSW 1 (`rest`, `res(ctx…)`); en MSW 2 eso es `http` y `HttpResponse`.
- **Contrato con Zod:** un esquema por respuesta y los tipos sacados con `z.infer`, una sola fuente de verdad. Toda respuesta se valida antes de usarse; si no cumple el contrato, se muestra un error.
- **Carga del menú** como unión discriminada: `cargando`, `error` (con el mensaje) o `listo` (con los productos).
- **Carrito** en un Context, como el `TodoContext` del apunte. Se recomienda `useReducer` con un reducer puro, fácil de testear.
- **Un solo módulo cliente** para las llamadas HTTP: ningún componente hace `fetch` directo.
- **Precios:** el front no calcula precios ni el total del pedido, los toma de la API. El subtotal del carrito es solo orientativo.
- **Estilos:** Bootstrap personalizado con SASS, con los colores de la cafetería definidos como variables antes de importar Bootstrap.
- **Accesibilidad:** cada control tiene su etiqueta, así los tests pueden buscarlo por rol y nombre como pide la teoría.

# Testing

Todo se desarrolla con TDD y sigue la pirámide de la teoría: muchos tests unitarios, menos de integración y pocos E2E.

| Nivel | Herramientas | Qué tiene que cubrir como mínimo |
|---|---|---|

| | | |
|---|---|---|
| Unitarios | Vitest | Funciones puras: el reducer del carrito, buscar el café que corresponde a la elección, el formato de precio y la lectura de la mesa desde la URL |
| Componentes | Vitest, React Testing Library, user-event, jest-dom | Selector de café (botón deshabilitado, precio por origen), pastelería y carrito (sumar, restar, vacío) |
| Integración | MSW 2 | Carga del menú; pedido completo verificando el cuerpo enviado a `POST /pedidos`; error 4xx que conserva el carrito; error 500 con reintento; API caída; respuesta que no cumple el contrato |
| E2E | Playwright | Al menos un recorrido feliz con front y API reales: elegir, agregar, confirmar y ver el pedido, con reset y seed antes de cada test |

Reglas para todos los niveles:

- Buscar elementos por rol, etiqueta o texto (`getByRole`, `getByLabelText`); `getByTestId` solo si no queda otra.
- Los handlers de MSW imitan la API del grupo: mismas rutas, mismos códigos y mismo formato de error.
- TDD visible en el historial: el commit del test en rojo va antes que el del código que lo pone en verde.
- `npm test` corre unitarios, componentes e integración sin necesitar la API levantada; `npm run test:e2e` corre los E2E.

# Entrega

Se entrega en el repositorio TP3 debe estar la carpeta `frontend/` y la API con los cambios.

**Flujo de trabajo:** una rama por funcionalidad, integrada a `main` con Pull Request se tiene que ver el flujo de TDD en los commits..

- **Participación:** como en todo el cuatrimestre, se mira la cantidad y la frecuencia de commits de cada integrante.
- **README:** cómo levantar la API y el front, cómo correr cada nivel de tests y el contrato de la API que usa el front.